TECHNIEK · ✨ SPARK — 1STE EN 2DE MIDDELBAAR · UITLEG BIJ DE OEFENBUNDEL

Algoritmen en computationeel denken

Bij elke oefening het antwoord en waarom het juist is. De nummers zijn dezelfde als in de oefenbundel.

- Dit blad is bedoeld om **samen na te lezen**, nadat je kind de bundel gemaakt heeft. Het hoort niet bij de bundel die je kind krijgt.
- Begin bij een fout antwoord met de vraag **hoe je kind eraan kwam**. Vaak zit de vergissing één stap eerder dan je denkt, en dan lees je hier samen die stap na.
- Staat er bij een oefening geen uitleg, dan zegt het antwoord zelf al genoeg.

1 Wat is een algoritme?

Een algoritme is gewoon een stappenplan. Vraag je af of de stappen vastliggen en of de volgorde ertoe doet.

1 Wat is een algoritme?

een stappenplan met instructies in een vaste volgorde

2 Algoritme of niet?

a) een algoritme · b) een algoritme · c) een algoritme

3 Een algoritme moet altijd op een computer draaien.

niet waar

Een recept en een stappenplan zijn evengoed algoritmen. Een computer is niet nodig, alleen vaste stappen in een vaste volgorde.

4 Wat is het verschil tussen een digitaal en een niet-digitaal algoritme? Geef van elk een voorbeeld.

Een niet-digitaal algoritme voer je zonder computer uit, bijvoorbeeld een stappenplan om een boterham met choco te smeren. Een digitaal algoritme laat je door een computer uitvoeren, bijvoorbeeld een lampje laten flikkeren met een programma.

5 In een algoritme mag je de stappen zomaar van volgorde wisselen.

niet waar

De volgorde is net de kern. Eerst boteren en dan snijden geeft een andere boterham dan omgekeerd.

6 Wat gebeurt er als één stap in een algoritme ontbreekt?

Het resultaat klopt meestal niet meer.

2 Een algoritme ontwerpen

Ontwerp in vaste stappen: het probleem benoemen, het met IPO uiteenleggen, het tekenen, het programmeren, het testen.

7 Hoe heet een schema met pijlen dat de stappen van een algoritme toont?

een flowchart of stroomdiagram

8 Waarvoor dient een flowchart?

Om de oplossing visueel voor te stellen, zodat je in één blik ziet hoe de stappen op elkaar volgen.

9 Waarvoor staat IPO bij het analyseren van een probleem?

input, process en output

10 Noem drie stappen die je volgt bij het ontwerpen van een algoritme.

Je formuleert de probleemstelling, je analyseert het probleem met IPO, en je test het algoritme uit.

11 In welke programmeertaal vertaal je volgens de vakfiche je algoritme?

Scratch

12 Waarom test je een algoritme uit, en wat doe je als het niet het juiste resultaat geeft?

Je test het om te zien of het echt doet wat je wilde. Geeft het niet het juiste resultaat, dan stuur je het bij.

13 Wat is de laatste stap bij het ontwerpen van een algoritme?

testen en bijsturen waar nodig

3 Computationeel denken

De vier principes zijn vier manieren om een groot probleem klein te maken. Zoek bij elke vraag welke je hier gebruikt.

14 Welk principe?

a) abstractie · b) decompositie · c) patroonherkenning

15 Welke vier principes van computationeel denken noemt de vakfiche?

Abstractie, decompositie, patroonherkenning en het algoritme.

16 Abstractie betekent zoveel mogelijk details toevoegen.

niet waar

Abstractie is net het omgekeerde: je laat weg wat er niet toe doet, zodat de kern overblijft.

17 Je schrijft een algoritme om de weg naar school uit te leggen. Noem een detail dat je weglaat, en zeg welk principe dat is.

De kleur van de huizen onderweg. Dat is abstractie: je laat weg wat er voor de weg niet toe doet.

18 Een spel heeft tien niveaus die sterk op elkaar lijken. Welk principe helpt hier, en wat win je ermee?

Patroonherkenning. Je hergebruikt de oplossing die je voor het eerste niveau al gevonden hebt, in plaats van tien keer opnieuw te beginnen.

19 Noem drie momenten waarop decompositie helpt.

Als een probleem te groot is voor één keer, als verschillende delen los van elkaar werken, en als je met meerdere mensen samenwerkt.

20 Wat is een voorbeeld van decompositie bij het maken van een spel?

apart werken aan de beweging, de punten en het geluid

4 Fouten en efficiëntie

Testen hoort bij het werk. Een algoritme dat je nooit uitvoerde, weet je ook niet of het werkt.

21 Hoe heet een fout in een programma?

een bug

22 Hoe heet het opsporen en oplossen van zo'n fout?

debuggen

23 Een programma moet een vierkant tekenen maar maakt maar drie zijden. Wat doe je?

Je debugt het algoritme: je zoekt de stap waar het fout gaat en past die aan.

24 Een fout in een algoritme vind je alleen door ernaar te kijken, nooit door het uit te voeren.

niet waar

Veel fouten zie je pas als je het programma laat lopen. Daarom test je altijd, ook als de code er juist uitziet.

25 Efficiënt betekent dat een algoritme zoveel mogelijk stappen heeft.

niet waar

Efficiënt betekent met zo weinig mogelijk stappen tot hetzelfde resultaat komen.

26 Waarom is een korter algoritme met dezelfde uitkomst meestal beter?

Er kunnen minder fouten in sluipen.

27 Noem drie dingen die je helpen om een efficiënt algoritme te schrijven.

Abstractie, decompositie en patroonherkenning.